



Turing machine

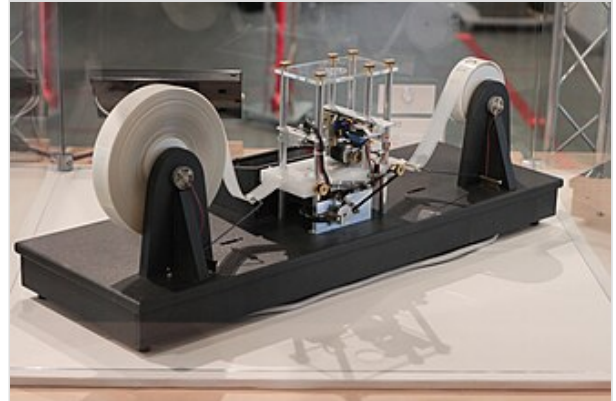
A **Turing machine** is a mathematical model of computation describing an abstract machine^[1] that manipulates symbols on a strip of tape according to a table of rules.^[2] Despite the model's simplicity, it is capable of implementing any computer algorithm.^[3]

The machine operates on an infinite^[4] memory tape divided into discrete cells,^[5] each of which can hold a single symbol drawn from a finite set of symbols called the alphabet of the machine. It has a "head" that, at any point in the machine's operation, is positioned over one of these cells, and a "state" selected from a finite set of states. At each step of its operation, the head reads the symbol in its cell. Then, based on the symbol and the machine's own present state, the machine writes a symbol into the same cell, and moves the head one step to the left or the right,^[6] or halts the computation. The choice of which replacement symbol to write, which direction to move the head, and whether to halt is based on a finite table that specifies what to do for each combination of the current state and the symbol that is read. As with a real computer program, it is possible for a Turing machine to go into an infinite loop which will never halt.

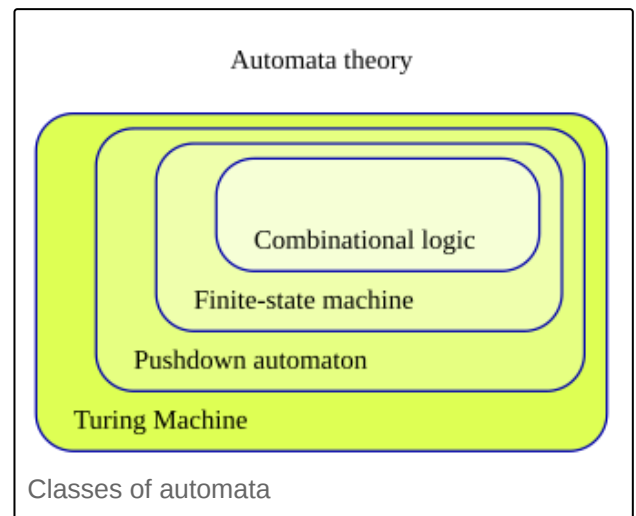
The Turing machine was invented in 1936 by Alan Turing,^{[7][11]} who called it an "a-machine" (automatic machine).^[12] It was Turing's doctoral advisor, Alonzo Church, who later coined the term "Turing machine" in a review.^[13] With this model, Turing was able to answer two questions in the negative:

- Does a machine exist that can determine whether any arbitrary machine on its tape is "circular" (e.g., freezes, or fails to continue its computational task)?
- Does a machine exist that can determine whether any arbitrary machine on its tape ever prints a given symbol?^{[14][15]}

Thus by providing a mathematical description of a very simple device capable of arbitrary computations, he was able to prove properties of computation in general—and in particular, the uncomputability of the Entscheidungsproblem, or 'decision problem' (whether every mathematical statement is provable or disprovable).^[16]



A physical Turing machine model constructed by Mike Davey. A true Turing machine would need to be provided more memory (tape) if and when required; physical models can only have a finite amount.



Turing machines proved the existence of fundamental limitations on the power of mechanical computation.^[3]

While they can express arbitrary computations, their minimalist design makes them too slow for computation in practice: real-world computers are based on different designs that, unlike Turing machines, use random-access memory.

Turing completeness is the ability for a model of computation or a system of instructions to simulate a Turing machine. A programming language that is Turing complete is theoretically capable of expressing all tasks accomplishable by computers; nearly all programming languages are Turing complete if the limitations of finite memory are ignored.

Overview

A Turing machine is an idealised model of a central processing unit (CPU) that controls all data manipulation done by a computer, with the canonical machine using sequential memory to store data. Typically, the sequential memory is represented as a tape of infinite length on which the machine can perform read and write operations.

In the context of formal language theory, a Turing machine (automaton) is capable of enumerating some arbitrary subset of valid strings of an alphabet. A set of strings which can be enumerated in this manner is called a recursively enumerable language. The Turing machine can equivalently be defined as a model that recognises valid input strings, rather than enumerating output strings.

Given a Turing machine M and an arbitrary string s , it is generally not possible to decide whether M will eventually produce s . This is due to the fact that the halting problem is unsolvable, which has major implications for the theoretical limits of computing.

A Turing machine that is able to simulate any other Turing machine is called a universal Turing machine (UTM, or simply a universal machine). Another mathematical formalism, lambda calculus, with a similar "universal" nature was introduced by Alonzo Church. Church's work intertwined with Turing's to form the basis for the Church–Turing thesis. This thesis states that Turing machines, lambda calculus, and other similar formalisms of computation do indeed capture the informal notion of effective methods in logic and mathematics and thus provide a model through which one can reason about an algorithm or "mechanical procedure" in a mathematically precise way without being tied to any particular formalism. Studying the abstract properties of Turing machines has yielded many insights into computer science, computability theory, and complexity theory.

Physical description

In his 1948 essay, "Intelligent Machinery", Turing wrote that his machine consists of:

...an unlimited memory capacity obtained in the form of an infinite tape marked out into squares, on each of which a symbol could be printed. At any moment there is one symbol in the machine; it is called the scanned symbol. The machine can alter the scanned symbol, and its behavior is in part determined by that symbol, but the symbols on the tape elsewhere do not

affect the behavior of the machine. However, the tape can be moved back and forth through the machine, this being one of the elementary operations of the machine. Any symbol on the tape may therefore eventually have an innings.^[17]

—Turing 1948^[18]

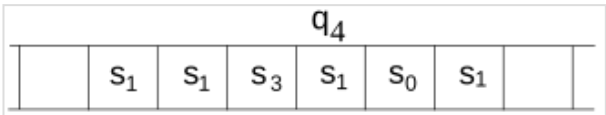
Description

The Turing machine mathematically models a machine that mechanically operates on a tape. On this tape are symbols, which the machine can read and write, one at a time, using a tape head. Operation is fully determined by a finite set of elementary instructions such as "in state 42, if the symbol seen is 0, write a 1; if the symbol seen is 1, change into state 17; in state 17, if the symbol seen is 0, write a 1 and change to state 6;" etc. In the original article ("On Computable Numbers, with an Application to the Entscheidungsproblem", see also references below), Turing imagines not a mechanism, but a person whom he calls the "computer", who executes these deterministic mechanical rules slavishly (or as Turing puts it, "in a desultory manner").

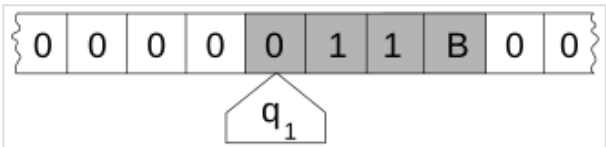
More explicitly, a Turing machine consists of:

- A *tape* divided into cells, one next to the other. Each cell contains a symbol from some finite alphabet. The alphabet contains a special blank symbol (here written as '0') and one or more other symbols. The tape is assumed to be arbitrarily extendable to the left and to the right, so that the Turing machine is always supplied with as much tape as it needs for its computation. Cells that have not been written before are assumed to be filled with the blank symbol. In some models the tape has a left end marked with a special symbol; the tape extends or is indefinitely extensible to the right.
- A *head* that can read and write symbols on the tape and move the tape left and right one (and only one) cell at a time. In some models the head moves and the tape is stationary.
- A *state register* that stores the state of the Turing machine, one of finitely many. Among these is the special *start state* with which the state register is initialised. These states, writes Turing, replace the "state of mind" a person performing computations would ordinarily be in.
- A finite *table*^[19] of instructions^[20] that, given the *state*(q_i) the machine is currently in *and* the *symbol*(a_j) it is reading on the tape (the symbol currently under the head), tells the machine to do the following *in sequence* (for the 5-tuple models):

1. Either erase or write a symbol (replacing a_j with a_{j1}).



The head is always over a particular square of the tape; only a finite stretch of squares is shown. The state of the machine (q_4) is shown over the square to be processed. (Drawing after Kleene (1952) p. 375.)



Here, the internal state (q_1) is shown inside the head, and the illustration describes the tape as being infinite and pre-filled with "0", the symbol serving as blank. The system's full state (its "complete configuration") consists of the internal state, any non-blank symbols on the tape (in this illustration "11B"), and the position of the head relative to those symbols including blanks, i.e. "011B". (Drawing after Minsky (1967) p. 121.)

2. Move the head (which is described by d_k and can have values: 'L' for one step left or 'R' for one step right or 'N' for staying in the same place).
3. Assume the same or a *new state* as prescribed (go to state q_{i1}).

In the 4-tuple models, erasing or writing a symbol (a_{j1}) and moving the head left or right (d_k) are specified as separate instructions. The table tells the machine to (ia) erase or write a symbol *or* (ib) move the head left or right, *and then* (ii) assume the same or a new state as prescribed, but not both actions (ia) and (ib) in the same instruction. In some models, if there is no entry in the table for the current combination of symbol and state, then the machine will halt; other models require all entries to be filled.

Every part of the machine (i.e. its state, symbol-collections, and used tape at any given time) and its actions (such as printing, erasing and tape motion) is *finite*, *discrete* and *distinguishable*; it is the unlimited amount of tape and runtime that gives it an unbounded amount of storage space.

Formal definition

Following Hopcroft & Ullman (1979),^[21] a (one-tape) Turing machine can be formally defined as a 7-tuple $M = \langle Q, \Gamma, b, \Sigma, \delta, q_0, F \rangle$ where

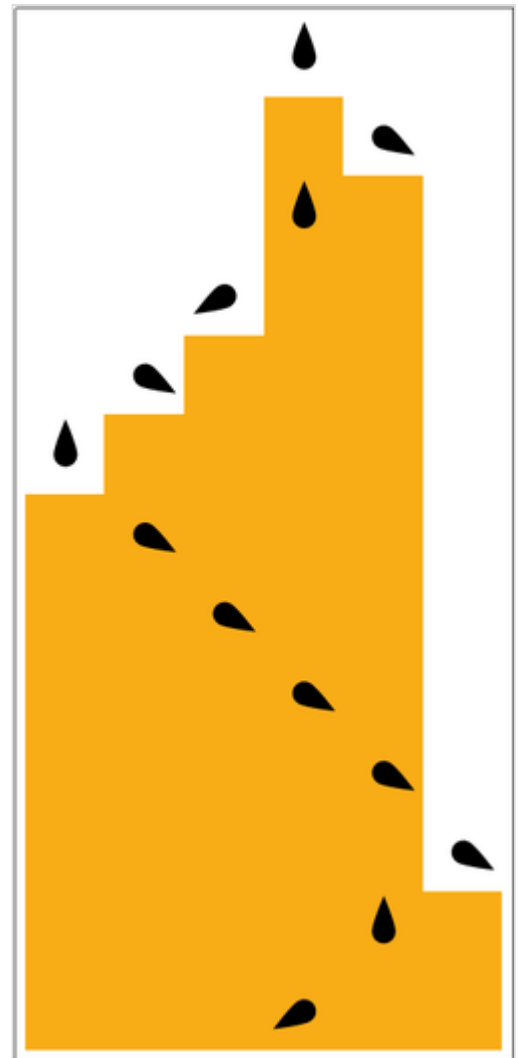
- Γ is a finite, non-empty set of *tape alphabet symbols*;
- $b \in \Gamma$ is the *blank symbol* (the only symbol allowed to occur on the tape infinitely often at any step during the computation);
- $\Sigma \subseteq \Gamma \setminus \{b\}$ is the set of *input symbols*, that is, the set of symbols allowed to appear in the initial tape contents;
- Q is a finite, non-empty set of *states*;
- $q_0 \in Q$ is the *initial state*;
- $F \subseteq Q$ is the set of *final states* or *accepting states*. The initial tape contents is said to be *accepted* by M if it eventually halts in a state from F .
- $\delta : (Q \setminus F) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is a partial function called the *transition function*, where L is left shift, R is right shift. If δ is not defined on the current state and the current tape symbol, then the machine halts;^[22] intuitively, the transition function specifies the next state transited from the current state, which symbol to overwrite the current symbol pointed by the head, and the next head movement.

A variant allows "no shift", say N, as a third element of the set of directions $\{L, R\}$.

The 7-tuple for the 3-state busy beaver looks like this (see more about this busy beaver at Turing machine examples):

- $Q = \{A, B, C, \text{HALT}\}$ (states);
- $\Gamma = \{0, 1\}$ (tape alphabet symbols);
- $b = 0$ (blank symbol);
- $\Sigma = \{1\}$ (input symbols);
- $q_0 = A$ (initial state);
- $F = \{\text{HALT}\}$ (final states);
- $\delta =$ see state-table below (transition function).

Initially all tape cells are marked with 0.



3-state Busy Beaver. Black icons represent location and state of head; square colors represent 1s (orange) and 0s (white); time progresses vertically from the top until the **HALT** state at the bottom.

State table for 3-state, 2-symbol busy beaver

Tape symbol	Current state A			Current state B			Current state C		
	Write symbol	Move tape	Next state	Write symbol	Move tape	Next state	Write symbol	Move tape	Next state
0	1	R	B	1	L	A	1	L	B
1	1	L	C	1	R	B	1	R	HALT

Additional details required to visualise or implement Turing machines

In the words of van Emde Boas (1990): "The set-theoretical object [his formal seven-tuple description similar to the above] provides only partial information on how the machine will behave and what its computations will look like."^[23]

For instance,

- There will need to be many decisions on what the symbols actually look like, and a failproof way of reading and writing symbols indefinitely.
- The shift left and shift right operations may shift the tape head across the tape, but when actually building a Turing machine it is more practical to make the tape slide back and forth under the head instead.
- The tape can be finite, and automatically extended with blanks as needed (which is closest to the mathematical definition), but it is more common to think of it as stretching infinitely at one or both ends and being pre-filled with blanks except on the explicitly given finite fragment the tape head is on (this is, of course, not implementable in practice). The tape *cannot* be fixed in length, since that would not correspond to the given definition and would seriously limit the range of computations the machine can perform to those of a linear bounded automaton if the tape was proportional to the input size, or finite-state machine if it was strictly fixed-length.

Alternative definitions

Definitions in literature sometimes differ slightly, to make arguments or proofs easier or clearer, but this is always done in such a way that the resulting machine has the same computational power. For example, the set could be changed from $\{L, R\}$ to $\{L, R, N\}$, where N ("None" or "No-operation") would allow the machine to stay on the same tape cell instead of moving left or right. This would not increase the machine's computational power.

The most common convention represents each "Turing instruction" in a "Turing table" by one of nine 5-tuples, per the convention of Turing/Davis (Turing (1936)^[24] and Davis (2000)^[25]):

(definition 1): ($q_i, S_j, S_k/E/N, L/R/N, q_m$)
 (current state q_i , symbol scanned S_j , print symbol S_k /erase E /none N ,
 move_tape_one_square left L /right R /none N , new state q_m)

Other authors (Minsky (1967),^[26] Hopcroft and Ullman (1979),^[27] Stone (1972),^[28] adopt a different convention, with new state q_m listed immediately after the scanned symbol S_j :

(definition 2): ($q_i, S_j, q_m, S_k/E/N, L/R/N$)
 (current state q_i , symbol scanned S_j , new state q_m , print symbol S_k /erase E /none N , move_tape_one_square left L /right R /none N)

For the remainder of this article "definition 1" (the Turing/Davis convention) will be used.

Example: state table for the 3-state 2-symbol busy beaver reduced to 5-tuples

Current state	Scanned symbol	Print symbol	Move tape	Final (i.e. next) state	5-tuples
A	0	1	R	B	(A, 0, 1, R, B)
A	1	1	L	C	(A, 1, 1, L, C)
B	0	1	L	A	(B, 0, 1, L, A)
B	1	1	R	B	(B, 1, 1, R, B)
C	0	1	L	B	(C, 0, 1, L, B)
C	1	1	N	H	(C, 1, 1, N, H)

In the following table, Turing's original model allowed only the first three lines that he called N1, N2, N3.^[29] He allowed for erasure of the "scanned square" by naming a 0th symbol S_0 = "erase" or "blank", etc. However, he did not allow for non-printing, so every instruction-line includes "print symbol S_k " or "erase".^[31] The abbreviations are Turing's.^[32] Subsequent to Turing's original paper in 1936–1937, machine-models have allowed all nine possible types of five-tuples:

	Current m-configuration (Turing state)	Tape symbol	Print-operation	Tape-motion	Final m-configuration (Turing state)	5-tuple	5-tuple comments	4-tuple
N1	q_i	S_j	Print(S_k)	Left L	q_m	(q_i, S_j, S_k, L, q_m)	"Blank" = S_0 , $1=S_1$, Etc.	
N2	q_i	S_j	Print(S_k)	Right R	q_m	(q_i, S_j, S_k, R, q_m)	"Blank" = S_0 , $1=S_1$, Etc.	
N3	q_i	S_j	Print(S_k)	None N	q_m	(q_i, S_j, S_k, N, q_m)	"Blank" = S_0 , $1=S_1$, Etc.	(q_i, S_j, S_k, q_m)
4	q_i	S_j	None N	Left L	q_m	(q_i, S_j, N, L, q_m)		(q_i, S_j, L, q_m)
5	q_i	S_j	None N	Right R	q_m	(q_i, S_j, N, R, q_m)		(q_i, S_j, R, q_m)
6	q_i	S_j	None N	None N	q_m	(q_i, S_j, N, N, q_m)	Direct "jump"	(q_i, S_j, N, q_m)
7	q_i	S_j	Erase	Left L	q_m	(q_i, S_j, E, L, q_m)		
8	q_i	S_j	Erase	Right R	q_m	(q_i, S_j, E, R, q_m)		
9	q_i	S_j	Erase	None N	q_m	(q_i, S_j, E, N, q_m)		(q_i, S_j, E, q_m)

Any Turing table (list of instructions) can be constructed from the above nine 5-tuples. For technical reasons, the three non-printing or "N" instructions (4, 5, 6) can usually be dispensed with. For examples see [Turing machine examples](#).

Less frequently the use of 4-tuples are encountered: these represent a further atomization of the Turing instructions.^[33]

The "state"

The word "state" used in context of Turing machines can be a source of confusion, as it can mean two things. Most commentators after Turing have used "state" to mean the name/designator of the current instruction to be performed—i.e. the contents of the state register. But Turing (1936) made a strong distinction between a record of what he called the machine's "m-configuration", and the machine's (or person's) "state of progress" through the computation—the current state of the total system. What Turing called "the state formula" includes both the current instruction and *all* the symbols on the tape:

Thus the state of progress of the computation at any stage is completely determined by the note of instructions and the symbols on the tape. That is, the *state of the system* may be described by a single expression (sequence of symbols) consisting of the symbols on the tape followed by Δ (which is supposed to not to appear elsewhere) and then by the note of instructions. This expression is called the "state formula".

—*The Undecidable*, pp. 139–140,^[34] emphasis added

Earlier in his paper Turing carried this even further: he gives an example where he placed a symbol of the current "m-configuration" —the instruction's label— beneath the scanned square, together with all the symbols on the tape.^[35] He calls it "the *complete configuration*".^[36] To print the "complete configuration" on one line, he places the state-label/m-configuration to the *left* of the scanned symbol.

A variant of this is seen in Kleene (1952) where Kleene shows how to write the Gödel number of a machine's "situation": he places the "m-configuration" symbol q_4 over the scanned square in roughly the center of the 6 non-blank squares on the tape (see the Turing-tape figure in this article) and puts it to the *right* of the scanned square.^[37] But Kleene refers to " q_4 " itself as "the machine state". Hopcroft and Ullman call this composite the "instantaneous description" and follow the Turing convention of putting the "current state" (instruction-label, m-configuration) to the *left* of the scanned symbol (p. 149), that is, the instantaneous description is the composite of non-blank symbols to the left, state of the machine, the current symbol scanned by the head, and the non-blank symbols to the right.

Example: total state of 3-state 2-symbol busy beaver after 3 "moves" (taken from example "run" in the figure below):

1A1

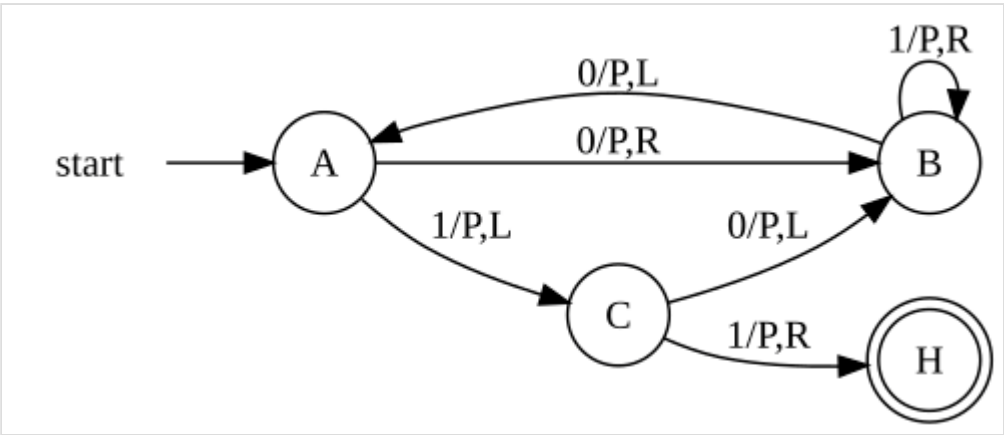
This means: after three moves the tape has ... 000110000 ... on it, the head is scanning the right-most 1, and the state is A. Blanks (in this case represented by "0"s) can be part of the total state as shown here: B01; the tape has a single 1 on it, but the head is scanning the 0 ("blank") to its left and the state is B.

"State" in the context of Turing machines should be clarified as to which is being described: the current instruction, or the list of symbols on the tape together with the current instruction, or the list of symbols on the tape together with the current instruction placed to the left of the scanned symbol or to the right of the scanned symbol.

"State" diagrams

The table for the 3-state busy beaver ("P" = print/write a "1")

Tape symbol	Current state A			Current state B			Current state C		
	Write symbol	Move tape	Next state	Write symbol	Move tape	Next state	Write symbol	Move tape	Next state
0	P	R	B	P	L	A	P	L	B
1	P	L	C	P	R	B	P	R	HALT



The "3-state busy beaver" Turing machine in a finite-state representation. Each circle represents a "state" of the table—an "m-configuration" or "instruction". "Direction" of a state *transition* is shown by an arrow. The label (e.g. 0/P,R) near the outgoing state (at the "tail" of the arrow) specifies the scanned symbol that causes a particular transition (e.g. 0) followed by a slash /, followed by the subsequent "behaviors" of the machine, e.g. "P print" then move tape "R right". No general accepted format exists. The convention shown is after McClusky (1965), Booth (1967), Hill, and Peterson (1974).

To the right: the above table as expressed as a "state transition" diagram.

Usually large tables are better left as tables (Booth, p. 74). They are more readily simulated by computer in tabular form (Booth, p. 74). However, certain concepts—e.g. machines with "reset" states and machines with repeating patterns (cf. Hill and Peterson p. 244ff)—can be more readily seen when viewed as a drawing.

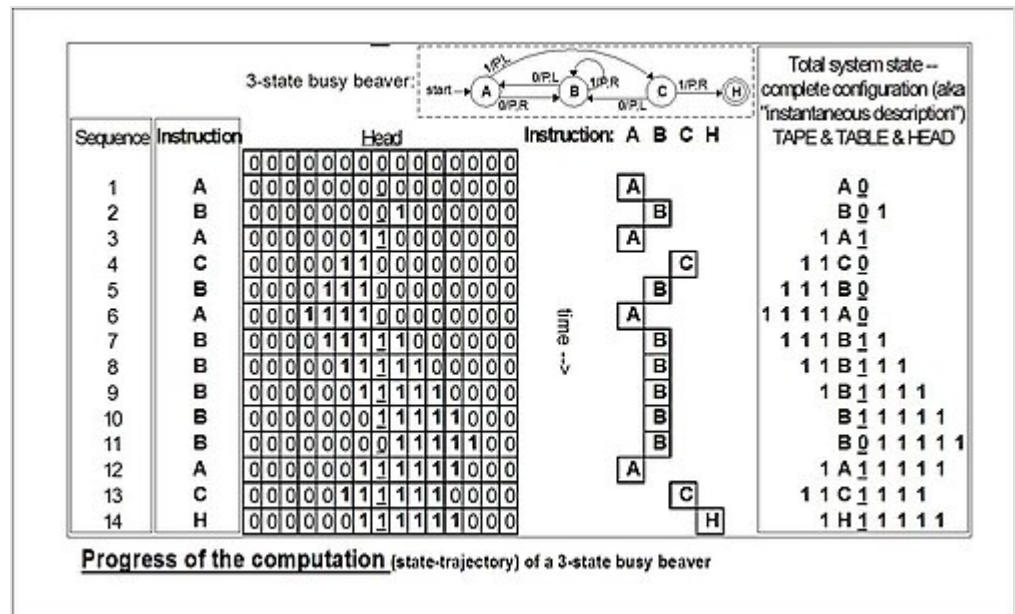
Whether a drawing represents an improvement on its table must be decided by the reader for the particular context.

The reader should again be cautioned that such diagrams represent a snapshot of their table frozen in time, *not* the course ("trajectory") of a computation *through* time and space. While every time the busy beaver machine "runs" it will always follow the same state-trajectory, this is not true for the "copy" machine that can be provided with variable input "parameters".

The diagram "progress of the computation" shows the three-state busy beaver's "state" (instruction) progress through its computation from start to finish. On the far right is the Turing "complete configuration" (Kleene "situation", Hopcroft–Ullman "instantaneous description") at each step. If the

machine were to be stopped and cleared to blank both the "state register" and entire tape, these "configurations" could be used to rekindle a computation anywhere in its progress.^[38]

Equivalent models



The evolution of the busy beaver's computation starts at the top and proceeds to the bottom.

Many machines that might be thought to have more computational capability than a simple universal Turing machine can be shown to have no more power (Hopcroft and Ullman p. 159, cf. Minsky (1967)). They might compute faster, perhaps, or use less memory, or their instruction set might be smaller, but they cannot compute more powerfully (i.e. more mathematical functions). (The Church–Turing thesis hypothesises this to be true for any kind of machine: that anything that can be "computed" can be computed by some Turing machine.)

A Turing machine is equivalent to a single-stack pushdown automaton (PDA) that has been made more flexible and concise by relaxing the last-in-first-out (LIFO) requirement of its stack. In addition, a Turing machine is also equivalent to a two-stack PDA with standard LIFO semantics, by using one stack to model the tape left of the head and the other stack for the tape to the right.

At the other extreme, some very simple models turn out to be Turing-equivalent, i.e. to have the same computational power as the Turing machine model.

Common equivalent models are the multi-tape Turing machine, multi-track Turing machine, machines with input and output, and the non-deterministic Turing machine (NDTM) as opposed to the deterministic Turing machine (DTM) for which the action table has at most one entry for each combination of symbol and state.

Read-only, right-moving Turing machines are equivalent to DFAs (as well as NFAs by conversion using the NFA to DFA conversion algorithm).

For practical and didactic intentions, the equivalent register machine can be used as a usual assembly programming language.

A relevant question is whether or not the computation model represented by concrete programming languages is Turing equivalent. While the computation of a real computer is based on finite states and thus not capable to simulate a Turing machine, programming languages themselves do not necessarily have this limitation. Kirner et al., 2009 have shown that among the general-purpose programming

languages some are Turing complete while others are not. For example, ANSI C is not Turing complete, as all instantiations of ANSI C (different instantiations are possible as the standard deliberately leaves certain behaviour undefined for legacy reasons) imply a finite-space memory. This is because the size of memory reference data types, called *pointers*, is accessible inside the language. However, other programming languages like Pascal do not have this feature, which allows them to be Turing complete in principle. It is just Turing complete in principle, as memory allocation in a programming language is allowed to fail, which means the programming language can be Turing complete when ignoring failed memory allocations, but the compiled programs executable on a real computer cannot.

Choice c-machines, oracle o-machines

Early in his paper (1936) Turing makes a distinction between an "automatic machine"—its "motion ... completely determined by the configuration" and a "choice machine":

...whose motion is only partially determined by the configuration ... When such a machine reaches one of these ambiguous configurations, it cannot go on until some arbitrary choice has been made by an external operator. This would be the case if we were using machines to deal with axiomatic systems.

—*The Undecidable*, p. 118^[36]

Turing (1936) does not elaborate further except in a footnote in which he describes how to use an a-machine to "find all the provable formulae of the [Hilbert] calculus" rather than use a choice machine. He "suppose[s] that the choices are always between two possibilities 0 and 1. Each proof will then be determined by a sequence of choices $i_1, i_2, \dots, i_n$ ($i_1 = 0$ or $1, i_2 = 0$ or $1, \dots, i_n = 0$ or 1), and hence the number $2^n + i_1 2^{n-1} + i_2 2^{n-2} + \dots + i_n$ completely determines the proof. The automatic machine carries out successively proof 1, proof 2, proof 3, ..." ^[39]

This is indeed the technique by which a deterministic (i.e., a-) Turing machine can be used to mimic the action of a nondeterministic Turing machine; Turing solved the matter in a footnote and appears to dismiss it from further consideration.

An oracle machine or o-machine is a Turing a-machine that pauses its computation at state "**o**" while, to complete its calculation, it "awaits the decision" of "the oracle"—an entity unspecified by Turing "apart from saying that it cannot be a machine" (Turing (1939)).^[40]

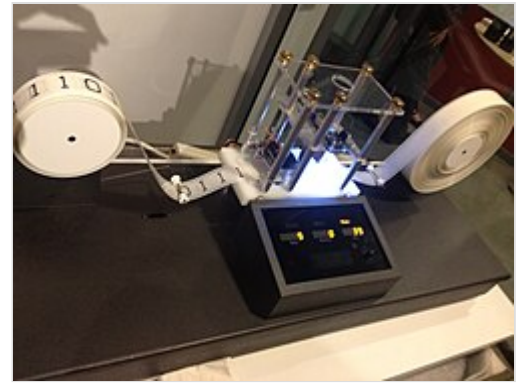
Universal Turing machines

As Turing wrote in *The Undecidable*, (italics added)^[41]:

It is possible to invent a *single machine* which can be used to compute *any* computable sequence. If this machine **U** is supplied with the tape on the beginning of which is written the string of quintuples separated by semicolons of some computing machine **M**, then **U** will compute the same sequence as **M**.

This finding is now taken for granted, but at the time (1936) it was considered astonishing. The model of computation that Turing called his "universal machine"—"U" for short—is considered by some^[42] to have been the fundamental theoretical breakthrough that led to the notion of the stored-program computer..

Turing's paper ... contains, in essence, the invention of the modern computer and some of the programming techniques that accompanied it.



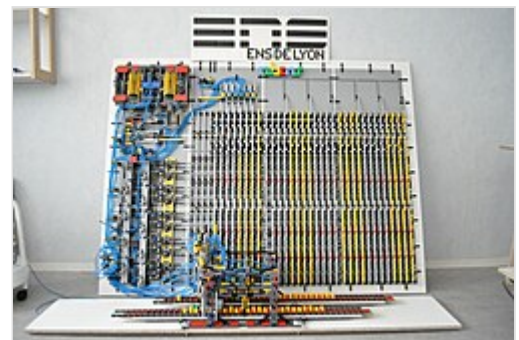
An implementation of a Turing machine

—Minsky (1967)^[43]

In terms of computational complexity, a multi-tape universal Turing machine need only be slower by logarithmic factor compared to the machines it simulates. This result was obtained in 1966 by F. C. Hennie and R. E. Stearns.^{[44][45]}

Comparison with real machines

Turing machines are more powerful than some other kinds of automata, such as finite-state machines and pushdown automata. According to the Church–Turing thesis, they are as powerful as real machines, and are able to execute any operation that a real program can. What is neglected in this statement is that, because a real machine can only have a finite number of *configurations*, it is nothing but a finite-state machine, whereas a Turing machine has an unlimited amount of storage space available for its computations.



A Turing machine realization using Lego pieces

There are a number of ways to explain why Turing machines are useful models of real computers:

- Anything a real computer can compute, a Turing machine can also compute. For example: "A Turing machine can simulate any type of subroutine found in programming languages, including recursive procedures and any of the known parameter-passing mechanisms" (Hopcroft and Ullman p. 157). A large enough FSA can also model any real computer, disregarding IO. Thus, a statement about the limitations of Turing machines will also apply to real computers.
- The difference lies only with the ability of a Turing machine to manipulate an unbounded amount of data. However, given a finite amount of time, a Turing machine (like a real machine) can only manipulate a finite amount of data.
- Like a Turing machine, a real machine can have its storage space enlarged as needed, by acquiring more disks or other storage media.
- Descriptions of real machine programs using simpler abstract models are often much more complex than descriptions using Turing machines. For example, a Turing machine describing an algorithm may have a few hundred states, while the equivalent deterministic

finite automaton (DFA) on a given real machine has quadrillions. This makes the DFA representation infeasible to analyze.

- Turing machines describe algorithms independent of how much memory they use. There is a limit to the memory possessed by any current machine, but this limit can rise arbitrarily in time. Turing machines allow us to make statements about algorithms which will (theoretically) hold forever, regardless of advances in *conventional* computing machine architecture.
- Algorithms running on Turing-equivalent abstract machines can have arbitrary-precision data types available and never have to deal with unexpected conditions (including, but not limited to, running out of memory).

Limitations

Computational complexity theory

A limitation of Turing machines is that they do not model the strengths of a particular arrangement well. For instance, modern stored-program computers are actually instances of a more specific form of abstract machine known as the random-access stored-program machine or RASP machine model. Like the universal Turing machine, the RASP stores its "program" in "memory" external to its finite-state machine's "instructions". Unlike the universal Turing machine, the RASP has an infinite number of distinguishable, numbered but unbounded "registers"—memory "cells" that can contain any integer (cf. Elgot and Robinson (1964), Hartmanis (1971), and in particular Cook-Rechow (1973); references at random-access machine). The RASP's finite-state machine is equipped with the capability for indirect addressing (e.g., the contents of one register can be used as an address to specify another register); thus the RASP's "program" can address any register in the register-sequence. The upshot of this distinction is that there are computational optimizations that can be performed based on the memory indices, which are not possible in a general Turing machine; thus when Turing machines are used as the basis for bounding running times, a "false lower bound" can be proven on certain algorithms' running times (due to the false simplifying assumption of a Turing machine). An example of this is binary search, an algorithm that can be shown to perform more quickly when using the RASP model of computation rather than the Turing machine model.

Interaction

In the early days of computing, computer use was typically limited to batch processing, i.e., non-interactive tasks, each producing output data from given input data. Computability theory, which studies computability of functions from inputs to outputs, and for which Turing machines were invented, reflects this practice.

Since the 1970s, interactive use of computers became much more common. In principle, it is possible to model this by having an external agent read from the tape and write to it at the same time as a Turing machine, but this rarely matches how interaction actually happens; therefore, when describing interactivity, alternatives such as I/O automata are usually preferred.

Comparison with the arithmetic model of computation

The arithmetic model of computation differs from the Turing model in two aspects:^[46]

- In the arithmetic model, every real number requires a single memory cell, whereas in the Turing model the storage size of a real number depends on the number of bits required to represent it.
- In the arithmetic model, every basic arithmetic operation on real numbers (addition, subtraction, multiplication and division) can be done in a single step, whereas in the Turing model the run-time of each arithmetic operation depends on the length of the operands.

Some algorithms run in polynomial time in one model but not in the other one. For example:

- The Euclidean algorithm runs in polynomial time in the Turing model, but not in the arithmetic model.
- The algorithm that reads n numbers and then computes 2^{2^n} by repeated squaring runs in polynomial time in the Arithmetic model, but not in the Turing model. This is because the number of bits required to represent the outcome is exponential in the input size.

However, if an algorithm runs in polynomial time in the arithmetic model, and in addition, the binary length of all involved numbers is polynomial in the length of the input, then it is always polynomial-time in the Turing model. Such an algorithm is said to run in strongly polynomial time.

History

Historical background: computational machinery

Robin Gandy (1919–1995)—a student of Alan Turing (1912–1954), and his lifelong friend—traces the lineage of the notion of "calculating machine" back to Charles Babbage (circa 1834) and actually proposes "Babbage's Thesis":

That the whole of development and operations of analysis are now capable of being executed by machinery.

—(italics in Babbage as cited by Gandy)^[47]

Gandy's analysis of Babbage's analytical engine describes the following five operations (cf. p. 52–53):

1. The arithmetic functions $+$, $-$, $\times$, where $-$ indicates "proper" subtraction: $x - y = 0$ if $y \geq x$.
2. Any sequence of operations is an operation.
3. Iteration of an operation (repeating n times an operation P).
4. Conditional iteration (repeating n times an operation P conditional on the "success" of test T).
5. Conditional transfer (i.e., conditional "goto").

Gandy states that "the functions which can be calculated by (1), (2), and (4) are precisely those which are Turing computable."^[48] He cites other proposals for "universal calculating machines" including those of Percy Ludgate (1909), Leonardo Torres Quevedo (1914),^{[49][50]} Maurice d'Ocagne (1922), Louis Couffignal (1933), Vannevar Bush (1936), Howard Aiken (1937). However:

... the emphasis is on programming a fixed iterable sequence of arithmetical operations. The fundamental importance of conditional iteration and conditional transfer for a general theory of calculating machines is not recognized...

—Gandy^[51]

The Entscheidungsproblem (the "decision problem"): Hilbert's tenth question of 1900

With regard to Hilbert's problems posed by the famous mathematician David Hilbert in 1900, an aspect of problem #10 had been floating about for almost 30 years before it was framed precisely. Hilbert's original expression for No. 10 is as follows:

10. Determination of the solvability of a Diophantine equation. Given a Diophantine equation with any number of unknown quantities and with rational integral coefficients: To devise a process according to which it can be determined in a finite number of operations whether the equation is solvable in rational integers. The Entscheidungsproblem [decision problem for first-order logic] is solved when we know a procedure that allows for any given logical expression to decide by finitely many operations its validity or satisfiability ... The Entscheidungsproblem must be considered the main problem of mathematical logic.

—quoted, with this translation and the original German, in Dershowitz and Gurevich, 2008^[52]

By 1922, this notion of "Entscheidungsproblem" had developed a bit, and H. Behmann stated that

... most general form of the Entscheidungsproblem [is] as follows:

A quite definite generally applicable prescription is required which will allow one to decide in a finite number of steps the truth or falsity of a given purely logical assertion ...

—Gandy^[53] quoting Behmann

Behmann remarks that ... the general problem is equivalent to the problem of deciding which mathematical propositions are true.

—*ibid.*

If one were able to solve the Entscheidungsproblem then one would have a "procedure for solving many (or even all) mathematical problems".

—*ibid.*^[54]

By the 1928 international congress of mathematicians, Hilbert "made his questions quite precise. First, was mathematics complete ... Second, was mathematics consistent ... And thirdly, was mathematics decidable?"^{[55][56]} The first two questions were answered in 1930 by Kurt Gödel at the very same meeting where Hilbert delivered his retirement speech (much to the chagrin of Hilbert); the third—the Entscheidungsproblem—had to wait until the mid-1930s.

The problem was that an answer first required a precise definition of "definite general applicable prescription", which Princeton professor Alonzo Church would come to call "effective calculability", and in 1928 no such definition existed. But over the next 6–7 years Emil Post developed his definition of a worker moving from room to room writing and erasing marks per a list of instructions,^[57] as did Church and his two students Stephen Kleene and J. B. Rosser by use of Church's lambda-calculus and Gödel's recursion theory (1934). Church's paper (published 15 April 1936) showed that the Entscheidungsproblem was indeed "undecidable"^[58] and beat Turing to the punch by almost a year (Turing's paper submitted 28 May 1936, published January 1937). In the meantime, Emil Post submitted a brief paper in the fall of 1936, so Turing at least had priority over Post. While Church refereed Turing's paper, Turing had time to study Church's paper and add an Appendix where he sketched a proof that Church's lambda-calculus and his machines would compute the same functions.

But what Church had done was something rather different, and in a certain sense weaker. ... the Turing construction was more direct, and provided an argument from first principles, closing the gap in Church's demonstration.

—Hodges^[9]

And Post had only proposed a definition of calculability and criticised Church's "definition", but had proved nothing.

Alan Turing's a-machine

In the spring of 1935, Turing as a young Master's student at King's College, Cambridge, took on the challenge; he had been stimulated by the lectures of the logician M. H. A. Newman "and learned from them of Gödel's work and the Entscheidungsproblem ... Newman used the word 'mechanical' ... In his obituary of Turing 1955 Newman writes:

To the question 'what is a "mechanical" process?' Turing returned the characteristic answer 'Something that can be done by a machine' and he embarked on the highly congenial task of analysing the general notion of a computing machine.

—Gandy^[59]

Gandy states that:

I suppose, but do not know, that Turing, right from the start of his work, had as his goal a proof of the undecidability of the Entscheidungsproblem. He told me that the 'main idea' of the paper came to him when he was lying in Grantchester meadows in the summer of 1935. The 'main idea' might have either been his analysis of computation or his realization that there was a universal machine, and so a diagonal argument to prove unsolvability.

—*ibid.* ^[60]

While Gandy believed that Newman's statement above is "misleading", this opinion is not shared by all. Turing had a lifelong interest in machines: "Alan had dreamt of inventing typewriters as a boy; [his mother] Mrs. Turing had a typewriter; and he could well have begun by asking himself what was meant by calling a typewriter 'mechanical'" (Hodges p. 96). While at Princeton pursuing his PhD, Turing built a Boolean-logic multiplier (see below). His PhD thesis, titled "Systems of Logic Based on Ordinals", contains the following definition of "a computable function":

It was stated above that 'a function is effectively calculable if its values can be found by some purely mechanical process'. We may take this statement literally, understanding by a purely mechanical process one which could be carried out by a machine. It is possible to give a mathematical description, in a certain normal form, of the structures of these machines. The development of these ideas leads to the author's definition of a computable function, and to an identification of computability with effective calculability. It is not difficult, though somewhat laborious, to prove that these three definitions [the 3rd is the λ -calculus] are equivalent.

—Turing (1939, p. 166)^[61]

Alan Turing invented the "a-machine" (automatic machine) in 1936.^[7] Turing submitted his paper on 31 May 1936 to the London Mathematical Society for its *Proceedings*,^[62] but it was published in early 1937 and offprints were available in February 1937.^[63] It was Turing's doctoral advisor, Alonzo Church, who later coined the term "Turing machine" in a review.^[13] With this model, Turing was able to answer two questions in the negative:

- Does a machine exist that can determine whether any arbitrary machine on its tape is "circular" (e.g., freezes, or fails to continue its computational task)?
- Does a machine exist that can determine whether any arbitrary machine on its tape ever prints a given symbol?^{[14][15]}

Thus by providing a mathematical description of a very simple device capable of arbitrary computations, he was able to prove properties of computation in general—and in particular, the uncomputability of the Entscheidungsproblem ('decision problem').^[16]

When Turing returned to the UK he ultimately became jointly responsible for breaking the German secret codes created by encryption machines called "The Enigma"; he also became involved in the design of the ACE (Automatic Computing Engine), "[Turing's] ACE proposal was effectively self-contained, and its roots lay not in the EDVAC [the USA's initiative], but in his own universal machine" (Hodges p. 318). Arguments still continue concerning the origin and nature of what has been named by Kleene (1952) Turing's Thesis. But what Turing *did prove* with his computational-machine model appears in his paper "On Computable Numbers, with an Application to the Entscheidungsproblem" (1937):

[that] the Hilbert Entscheidungsproblem can have no solution ... I propose, therefore to show that there can be no general process for determining whether a given formula U of the functional calculus K is provable, i.e. that there can be no machine which, supplied with any one U of these formulae, will eventually say whether U is provable.

—from Turing's paper as reprinted in *The Undecidable*, p. 145^[64]

Turing's example (his second proof): If one is to ask for a general procedure to tell us: "Does this machine ever print 0", the question is "undecidable".

1937–1970: The "digital computer", the birth of "computer science"

In 1937, while at Princeton working on his PhD thesis, Turing built a digital (Boolean-logic) multiplier from scratch, making his own electromechanical relays (Hodges p. 138). "Alan's task was to embody the logical design of a Turing machine in a network of relay-operated switches ..." (Hodges p. 138). While Turing might have been just initially curious and experimenting, quite-earnest work in the same direction was going in Germany (Konrad Zuse (1938)), and in the United States (Howard Aiken) and George Stibitz (1937); the fruits of their labors were used by both the Axis and Allied militaries in World War II (cf. Hodges p. 298–299). In the early to mid-1950s Hao Wang and Marvin Minsky reduced the Turing machine to a simpler form (a precursor to the Post–Turing machine of Martin Davis); simultaneously European researchers were reducing the new-fangled electronic computer to a computer-like theoretical object equivalent to what is now being called a "Turing machine". In the late 1950s and early 1960s, the coincidentally parallel developments of Melzak and Lambek (1961), Minsky (1961), and Shepherdson and Sturgis (1961) carried the European work further and reduced the Turing machine to a more friendly, computer-like abstract model called the counter machine; Elgot and Robinson (1964), Hartmanis (1971), Cook and Reckhow (1973) carried this work even further with the register machine and random-access machine models—but basically all are just multi-tape Turing machines with an arithmetic-like instruction set.

1970–present: as a model of computation

Today, the counter, register and random-access machines and their sire the Turing machine continue to be the models of choice for theorists investigating questions in the theory of computation. In particular, computational complexity theory makes use of the Turing machine:

Depending on the objects one likes to manipulate in the computations (numbers like nonnegative integers or alphanumeric strings), two models have obtained a dominant position in machine-based complexity theory:

the off-line multitape Turing machine..., which represents the standard model for string-oriented computation, and the *random access machine (RAM)* as introduced by Cook and Reckhow ..., which models the idealised Von Neumann-style computer.

—Van Emde Boas (1990)^[65]

Only in the related area of analysis of algorithms this role is taken over by the RAM model.

—Van Emde Boas (1990)^[66]

See also

- [Arithmetical hierarchy](#)
- [Bekenstein bound](#), showing the impossibility of infinite-tape Turing machines of finite size and bounded energy
- [BlooP and Floop](#)
- [Chaitin's constant](#) or [Omega \(computer science\)](#) for information relating to the halting problem
- [Calculus ratiocinator](#)
- [Chinese room](#)
- [Conway's Game of Life](#), a Turing-complete cellular automaton
- [Digital infinity](#)
- [The Emperor's New Mind](#)
- [Enumerator \(computer science\)](#)
- [Genetix](#)
- [Gödel, Escher, Bach: An Eternal Golden Braid](#), a famous book that discusses, among other topics, the Church–Turing thesis
- [Halting problem](#), for more references
- [Harvard architecture](#)
- [Imperative programming](#)
- [Langton's ant](#) and [Turmites](#), simple two-dimensional analogues of the Turing machine
- [List of things named after Alan Turing](#)
- [Modified Harvard architecture](#)
- [Quantum Turing machine](#)
- [Claude Shannon](#), another leading thinker in information theory
- [Turing machine examples](#)
- [Turing tarpit](#), any computing system or language that, despite being Turing complete, is generally considered useless for practical computing
- [Unorganised machine](#), for Turing's very early ideas on neural networks
- [Von Neumann architecture](#)

Notes

1. Minsky (1967, p. 107) "In his 1936 paper, A. M. Turing defined the class of abstract machines that now bear his name. A Turing machine is a finite-state machine associated with a special kind of environment —its tape— in which it can store (and later recover) sequences of symbols," also Stone (1972, p. 8) where the word "machine" is in quotation marks.
2. Stone (1972, p. 8) states: "This "machine" is an abstract mathematical model", also cf. Sipser (2012, p. 165ff) that describes the "Turing machine model". Rogers (1987, p. 13) refers to "Turing's characterization", Boolos, Burgess & Jeffrey (2002, p. 25) refers to a "specific kind of idealized machine".
3. Sipser (2012, p. 165) observes that "[a] Turing machine can do everything that a real computer can do. Nonetheless, even a Turing machine cannot solve certain problems. In a very real sense, these problems are beyond the theoretical limits of computation."
4. Cf. Sipser (2012, p. 165). Also, Rogers (1987, p. 13) describes "a paper tape of infinite length in both directions". Minsky (1967, p. 118) states "The tape is regarded as infinite in both directions". Boolos, Burgess & Jeffrey (2002, p. 25) include the possibility of "there is someone stationed at each end to add extra blank squares as needed".
5. Cf. Rogers (1987, p. 13). Other authors use the word "square" e.g. Boolos, Burgess & Jeffrey (2002, p. 35), Minsky (1967, p. 117), Penrose (1989, p. 37).
6. Boolos, Burgess & Jeffrey (2002, p. 25) illustrate the machine as moving along the tape. Penrose (1989, pp. 36–37) describes himself as "uncomfortable" with an infinite tape observing that it "might be hard to shift!"; he "prefer[s] to think of the tape as representing some external environment through which our finite device can move" and after observing that the "'movement' is a convenient way of picturing things" and then suggests that "the device receives all its input from this environment. Some variations of the Turing machine model also allow the head to stay in the same position instead of moving or halting."
7. Hodges 2012.
8. Hodges 1983, p. 93.
9. Hodges 1983, p. 112.
10. Hodges 1983, p. 129.
11. The idea came to him in mid-1935 (perhaps, see more in the History section) after a question posed by M. H. A. Newman in his lectures: "Was there a definite method, or as Newman put it, a "mechanical process" which could be applied to a mathematical statement, and which would come up with the answer as to whether it was provable".^[8] Turing submitted his paper on 31 May 1936 to the London Mathematical Society for its *Proceedings*,^[9] but it was published in early 1937 and offprints were available in February 1937.^[10]
12. See footnote in Davis (2000, p. 151)
13. See note in forward Tyler & Emderton (2019)
14. Turing 1936 in Davis (2004, pp. 132–134); Turing's definition of "circular" is found on page 119.
15. Turing 1936, pp. 230–265.
16. Turing 1936 in Davis (2004, p. 145)
17. See the definition of "innings" on Wiktionary
18. Turing (1968, pp. 3–4)
19. Occasionally called an *action table* or *transition function*.
20. Usually quintuples [5-tuples]: $q_i a_j \rightarrow q_{i1} a_{j1} d_k$, but sometimes quadruples [4-tuples].

21. [Hopcroft & Ullman 1979](#), p. 148.
22. p.149; in particular, Hopcroft and Ullman assume that δ is undefined on all states from F
23. [Van Emde Boas 1990](#), p. 6.
24. [Davis 2004](#), pp. 126–127.
25. [Davis 2000](#), p. 152.
26. [Minsky 1967](#), p. 119.
27. [Hopcroft & Ullman 1979](#), p. 158.
28. [Stone 1972](#), p. 9.
29. Cf. Turing in [Davis \(2004](#), p. 126)
30. [Davis 2004](#), p. 300.
31. Cf. footnote 12 in [Post \(1947\)](#)^[30]
32. [Davis 2004](#), p. 119.
33. Cf. [Post \(1947\)](#), [Boolos & Jeffrey \(1999\)](#), [Davis, Sigal & Weyuker \(1994\)](#). Also see more at [Post–Turing machine](#).
34. [Davis 2004](#), pp. 139–140.
35. [Davis 2004](#), p. 121.
36. [Davis 2004](#), p. 118.
37. [Kleene 1952](#), pp. 374–375.
38. Cf. [Turing \(1936\)](#)^[34]
39. Footnote ‡ in [Davis \(2004](#), p. 138)
40. [Davis 2004](#), pp. 166–168.
41. [Davis 2004](#), p. 128.
42. [Davis 2000](#).
43. [Minsky 1967](#), p. 104.
44. [Hennie & Stearns 1966](#).
45. [Arora & Barak 2009](#), theorem 1.9.
46. [Grötschel, Lovász & Schrijver 1993](#), p. 32.
47. [Gandy 1995](#), p. 54.
48. [Gandy 1995](#), p. 53.
49. [Torres Quevedo 1914](#).
50. [Torres Quevedo 1915](#).
51. [Gandy 1995](#), p. 55.
52. [Dershowitz & Gurevich 2008](#).
53. [Gandy 1995](#), p. 57.
54. [Gandy 1995](#), p. 92.
55. [Hodges 1983](#), p. 91.
56. [Hawking 2005](#), p. 1121.
57. [Post 1936](#).
58. The narrower question posed in [Hilbert's tenth problem](#), about [Diophantine equations](#), remained unresolved until 1970, when the relationship between [recursively enumerable sets](#) and [Diophantine sets](#) was finally laid bare.
59. [Gandy 1995](#), p. 74.
60. [Gandy 1995](#), p. 76.
61. [Davis 2004](#), p. 160.
62. Cf. [Hodges \(1983](#), p. 112)

63. Cf. Hodges (1983, p. 129)
64. Davis 2004, p. 145.
65. Van Emde Boas 1990, p. 4.
66. Van Emde Boas 1990, p. 16.

References

Primary literature, reprints, and compilations

- Church, Alonzo (23 April 2019). Burge, Tyler; Enderton, Herbert (eds.). *The Collected Works of Alonzo Church* (<https://mitpress.mit.edu/books/collected-works-alonzo-church>). Cambridge, MA, US: MIT Press. ISBN 978-0-262-02564-5.
- Copeland, Jack, ed. (2004). *The Essential Turing: Seminal Writings in Computing, Logic, Philosophy, Artificial Intelligence, and Artificial Life plus The Secrets of Enigma*. Oxford, UK: Clarendon Press (Oxford University Press). ISBN 0-19-825079-7. "Contains Turing papers, a draft letter to Emil Post on criticism of "Turing's convention", and Donald W. Davies' *Corrections to Turing's Universal Computing Machine*"
- Davis, Martin, ed. (2004) [1965]. *The Undecidable: Basic Papers on Undecidable Propositions, Unsolvable Problems and Computable Functions*. Mineola, NY: Dover Publ. ISBN 978-0486432281.
- Hennie, F. C.; Stearns, R. E. (1966). "Two-tape simulation of multitape Turing machines". *Journal of the ACM*. **13** (4): 533–546.
- Post, Emil (1936). "Finite Combinatory Processes—Formulation 1". *Journal of Symbolic Logic*. **1**: 103–105. "Reprinted in Davis (2004, p. 289ff)"
- Post, Emil (1947). "Recursive Unsolvability of a Problem of Thue". *Journal of Symbolic Logic*. **12**: 1–11. "Reprinted in Davis (2004, p. 293ff); appendix includes corrections and commentary on Turing's 1936–1937 paper. In particular see the footnotes 11 with corrections to the universal computing machine coding and footnote 14 with comments on Turing's first and second proofs."
- Turing, A. M. (1936). "On Computable Numbers, with an Application to the Entscheidungsproblem" (<https://www.cs.ox.ac.uk/activities/ieg/e-library/sources/tp2-ie.pdf>) (PDF). *Proceedings of the London Mathematical Society*. 2. **42** (published 1937): 230–265. doi:10.1112/plms/s2-42.1.230 (<https://doi.org/10.1112%2Fplms%2Fs2-42.1.230>). S2CID 73712 (<https://api.semanticscholar.org/CorpusID:73712>).
- Turing, A. M. (1938). "On Computable Numbers, with an Application to the Entscheidungsproblem: A correction". *Proceedings of the London Mathematical Society*. 2. **43** (6): 544–546. doi:10.1112/plms/s2-43.6.544 (<https://doi.org/10.1112%2Fplms%2Fs2-43.6.544>). "Reprinted in Davis (2004, pp. 115–154)"
- Turing, A.M. (1939). "Systems of Logic Based on Ordinals". *Proceedings of the London Mathematical Society*. s2-45 (1): 161–228. doi:10.1112/plms/s2-45.1.161 (<https://doi.org/10.1112%2Fplms%2Fs2-45.1.161>). hdl:21.11116/0000-0001-91CE-3 (<https://hdl.handle.net/21.11116%2F0000-0001-91CE-3>).
- Turing, Alan (1968) [1948]. "Intelligent Machinery". In Evans, C. R.; Robertson, A. D. J. (eds.). *Cybernetics: Key Papers*. Baltimore: University Park Press. p. 31. Reprint: Turing, A. M. (1996). "Intelligent Machinery, A Heretical Theory" (<https://doi.org/10.1093%2Fphilmat%2>

Computability theory

- Boolos, George; Jeffrey, Richard (1999) [1974]. *Computability and Logic* (https://archive.org/details/computabilitylog0000bool_r8y9) (3rd ed.). Cambridge UK: Cambridge University Press. ISBN 0-521-20402-X.
- Boolos, George; Burgess, John P.; Jeffrey, Richard (2002) [1989]. *Computability and Logic* (4th ed.). Cambridge UK: Cambridge University Press. ISBN 0-521-00758-5. "Some parts have been significantly rewritten by Burgess. Presentation of Turing machines in context of Lambek "abacus machines" (cf. Register machine) and recursive functions, showing their equivalence."
- Booth, Taylor L. (1967). *Sequential Machines and Automata Theory*. New York: John Wiley and Sons, Inc. "Graduate-level engineering text; ranges over a wide variety of topics. Chapter IX "Turing Machines" includes some recursion theory."
- Davis, Martin (2000). *The Universal Computer: The Road from Leibniz to Turing*. W. W. Norton & Company. ISBN 0-393-04785-7. Reprinted as *Engines of Logic: Mathematicians and the Origin of the Computer*. New York: Norton. 2001. ISBN 9780393322293.
- Davis, Martin; Sigal, Ron; Weyuker, Elaine J. (1994). *Computability, Complexity, and Languages and Logic: Fundamentals of Theoretical Computer Science* (2nd ed.). San Diego: Academic Press, Harcourt, Brace & Company. ISBN 0-12-206382-1.
- Grötschel, Martin; Lovász, László; Schrijver, Alexander (1993). *Geometric algorithms and combinatorial optimization*. Algorithms and Combinatorics. Vol. 2 (2nd ed.). Berlin: Springer-Verlag. doi:10.1007/978-3-642-78240-4 (<https://doi.org/10.1007%2F978-3-642-78240-4>). ISBN 978-3-642-78242-8. MR 1261419 (<https://mathscinet.ams.org/mathscinet-getitem?mr=1261419>).
- Hennie, Fredrick (1977). *Introduction to Computability*. Reading, Mass.: Addison–Wesley. QA248.5H4 1977. "On pages 90–103 Hennie discusses the UTM with examples and flow-charts, but no actual 'code'."
- Hopcroft, John; Ullman, Jeffrey D. (1979). *Introduction to Automata Theory, Languages, and Computation* (1st ed.). Reading, Mass.: Addison–Wesley. ISBN 0-201-02988-X. "Centered around the issues of machine-interpretation of "languages", NP-completeness, etc."
- Hopcroft, John E.; Motwani, Rajeev; Ullman, Jeffrey D. (2001). *Introduction to Automata Theory, Languages, and Computation* (2nd ed.). Reading, Mass.: Addison–Wesley. ISBN 0-201-44124-1.
- Kleene, Stephen (1952). *Introduction to Metamathematics* (10th impression (with corrections of 6th reprint 1971) ed.). Amsterdam, Netherlands: North–Holland Publishing Company. "Graduate level text; most of Chapter XIII "Computable functions" is on Turing machine proofs of computability of recursive functions, etc."
- Knuth, Donald E. (1973). *The Art of Computer Programming, Vol. 1: Fundamental Algorithms* (2nd ed.). Reading, Mass.: Addison–Wesley Publishing Company. "With reference to the role of Turing machines in the development of computation see 1.4.5 "History and Bibliography" pp. 225ff and 2.6 "History and Bibliography" pp. 456ff."
- Manna, Zohar (1974). *Mathematical Theory of Computation* (Reprinted, Dover, 2003 ed.). McGraw–Hill. ISBN 978-0-486-43238-0.

- Minsky, Marvin (1967). *Computation: Finite and Infinite Machines*. N.J.: Prentice–Hall, Inc. "See Chapter 8, Section 8.2 "Unsolvability of the Halting Problem." "
- Papadimitriou, Christos (1993). *Computational Complexity* (1st ed.). Addison Wesley. ISBN 0-201-53082-1. "Chapter 2: Turing machines, pp. 19–56."
- Torres Quevedo, Leonardo (1914). "Ensayos sobre Automática – Su definición. Extensión teórica de sus aplicaciones". *Revista de la Academia de Ciencias Exactas* (in Spanish). **12**: 391–418.
- Torres Quevedo, Leonardo (1915). "Essais sur l'Automatique – Sa définition. Étendue théorique de ses applications" (<https://diccan.com/dicoport/Torres.htm>). *Revue Générale des Sciences Pures et Appliquées* (in French). **2**: 601–611.
- Rogers, Hartley (1987) [1967]. *Theory of Recursive Functions and Effective Computability* (Paperback (original 1967 McGraw–Hill) ed.). Cambridge, MA: The MIT Press. ISBN 0-262-68052-1.
- Sipser, Michael (2012) [1997]. "Chapter 3: The Church–Turing Thesis". *Introduction to the Theory of Computation* (Third ed.). Cengage Learning. pp. 165–192. ISBN 978-1-133-18779-0.
- Stone, Harold S. (1972). *Introduction to Computer Organization and Data Structures* (1st ed.). New York: McGraw–Hill Book Company. ISBN 0-07-061726-0.
- Van Emde Boas, Peter (1990). "Machine Models and Simulations". In Jan van Leeuwen (ed.). *Handbook of Theoretical Computer Science, Volume A: Algorithms and Complexity*. The MIT Press / Elsevier. pp. 3–66. ISBN 0-444-88071-2. "Library of Congress call number: QA76.H279 1990"

Church's thesis

- Dershowitz, Nachum; Gurevich, Yuri (September 2008). "A natural axiomatization of computability and proof of Church's Thesis" (<http://research.microsoft.com/en-us/um/people/gurevich/Opera/188.pdf>) (PDF). *Bulletin of Symbolic Logic*. **14** (3). Retrieved 12 December 2025.

Other

- Arora, Sanjeev; Barak, Boaz (2009). "Sections 1.4 and 1.7". *Complexity Theory: A Modern Approach* (<http://www.cs.princeton.edu/theory/complexity/>). Cambridge University Press. ISBN 978-0-521-42426-4.
- Gandy, Robin (1995). "The Confluence of Ideas in 1936". In Herken, Rolf (ed.). *The Universal Turing Machine—A Half-Century Survey*. Springer Verlag. pp. 51–102. ISBN 978-3-211-82637-9.
- Hawking, Stephen, ed. (2005). *God Created the Integers: The Mathematical Breakthroughs that Changed History*. Philadelphia: Running Press. ISBN 978-0-7624-1922-7. "Includes Turing's 1936–1937 paper with commentary and biography by Hawking"
- Hodges, Andrew (1983). "The Spirit of Truth". *Alan Turing: The Enigma*. New York: Simon and Schuster. "Cf. Chapter "The Spirit of Truth" for a history leading to, and a discussion of, his proof."

- Hodges, Andrew (2012). *Alan Turing: The Enigma* (The Centenary ed.). Princeton University Press. ISBN 978-0-691-15564-7.
- Kantorovitz, Isaiah Pinchas (1 December 2005). "A note on Turing machine computability of rule driven systems". *SI GACT News*. **36** (4): 109–110. doi:10.1145/1107523.1107525 ([http s://doi.org/10.1145%2F1107523.1107525](http://doi.org/10.1145%2F1107523.1107525)). S2CID 31117713 (<https://api.semanticscholar.org/CorpusID:31117713>).
- Kirner, Raimund; Zimmermann, Wolf; Richter, Dirk (October 2009). "On Undecidability Results of Real Programming Languages" ([http://researchprofiles.herts.ac.uk/portal/en/publications/on-undecidability-results-of-real-programming-languages\(d3f3aac0-d9da-4756-a421-b4f9ae0cf95f\).html](http://researchprofiles.herts.ac.uk/portal/en/publications/on-undecidability-results-of-real-programming-languages(d3f3aac0-d9da-4756-a421-b4f9ae0cf95f).html)). 15. *Kolloquium Programmiersprachen und Grundlagen der Programmierung (KPS'09)*. Maria Taferl, Austria.
- Penrose, Roger (1989). *The Emperor's New Mind: Concerning Computers, Minds, and the Laws of Physics*. Oxford and New York: Oxford University Press. ISBN 0-19-851973-7. "1990 corrections"
- Petzold, Charles (2008). *The Annotated Turing* (<http://www.theannotatedturing.com/>). John Wiley & Sons, Inc. ISBN 0-470-22905-5.
- Wang, Hao (1957). "A variant to Turing's theory of computing machines". *Journal of the Association for Computing Machinery (KACM)*. **4**: 63–92.

External links

- "Turing machine" (https://www.encyclopediaofmath.org/index.php?title=Turing_machine), *Encyclopedia of Mathematics*, EMS Press, 2001 [1994]
 - "Turing Machines" (<https://plato.stanford.edu/entries/turing-machine/>) entry by De Mol, Liesbeth in the *Stanford Encyclopedia of Philosophy*, Summer 2025
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=Turing_machine&oldid=1330296720"